

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: Examine intermediate code generation techniques to enhance code optimization (BL4)

Assessment Tool Weightage: 20%

QUESTIONS:

Total Marks:50

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Code of Conduct:

I Hemalatha.C (192524475) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:Hemalatha.C

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Q1. E-commerce Order Processing System.

Concepts Covered: Declarations, Assignment Statements, Intermediate Code

Problem Statement

An e-commerce platform processes customer orders. Variables include product price, quantity, tax, and total amount.

Task

1. Write declarations for all variables.
2. Convert the following high-level code into three-address intermediate code:
$$\text{total} = \text{price} * \text{quantity}$$
$$\text{tax_amount} = \text{total} * \text{tax}$$
$$\text{final_amount} = \text{total} + \text{tax_amount}$$
3. Optimize the intermediate code (if possible).

Q2. Banking Loan Approval System

Concepts Covered: Boolean Expressions, Backpatching

Problem Statement

A bank evaluates loan eligibility based on:

- Salary > 30,000
- Credit Score > 700

Task

1. Convert the condition into Boolean expression intermediate code.
2. Generate flow of control using backpatching technique.
3. Represent using labelled instructions.

Q3. Traffic Signal Control System

Concepts Covered: Case Statements

Problem Statement

A smart traffic system changes signals based on input:

```
switch(signal)
case 1: RED
```

case 2: YELLOW

case 3: GREEN

Task

1. Convert the case statement into intermediate code.
2. Use jump tables or conditional jumps.
3. Explain efficiency of your approach.

Q4. Online Payment Gateway

Concepts Covered: Procedure Calls

Problem Statement

A payment system calls functions:

validateCard()
processPayment()
generateReceipt()

Task

1. Generate intermediate code for procedure calls.
2. Show:
 - o Parameter passing
 - o Call and return sequence
3. Use stack-based representation.

Q5. Smart Home Automation System

Concepts Covered: Boolean Expressions, Assignment, Backpatching

Problem Statement

A smart home system turns ON AC if:

- Temperature > 30 AND Room Occupied

Task

1. Write Boolean expressions.
2. Convert into intermediate code with short-circuit evaluation.
3. Apply backpatching to handle jumps.
4. Assign final output to variable AC_status.

Q1. E-Commerce Order Processing System

1. Declarations

```
float price;  
int quantity;  
float tax;  
float total;  
float tax_amount;  
float final_amount;
```

2. Three-Address Code

Given:

```
total = price * quantity  
tax_amount = total * tax  
final_amount = total + tax_amount
```

Three-address intermediate code:

```
t1 = price * quantity  
total = t1
```

```
t2 = total * tax  
tax_amount = t2
```

```
t3 = total + tax_amount  
final_amount = t3
```

3. Optimized Intermediate Code

The temporary variables can be reduced because the results can be directly assigned:

```
total = price * quantity  
tax_amount = total * tax  
final_amount = total + tax_amount
```

Optimization: Unnecessary temporary variables t1, t2, and t3 are removed.

Q2. Banking Loan Approval System

Conditions

Salary > 30000

Credit Score > 700

Assume both conditions must be true.

Boolean expression:

Salary > 30000 AND CreditScore > 700

1. Boolean Intermediate Code

Using short-circuit evaluation:

**if Salary > 30000 goto L1
goto Lfalse**

**L1:
if CreditScore > 700 goto Ltrue
goto Lfalse**

**Ltrue:
Loan = Approved
goto Lend**

**Lfalse:
Loan = Rejected**

Lend:

2. Backpatching

Initially, the true and false jumps contain incomplete targets.

**if Salary > 30000 goto _
goto _**

**if CreditScore > 700 goto _
goto _**

After backpatching:

**if Salary > 30000 goto L1
goto Lfalse**

L1:

```
if CreditScore > 700 goto Ltrue  
goto Lfalse
```

The blank jump targets are replaced with the appropriate labels.

3. Labelled Instructions

L1:

```
if CreditScore > 700 goto Ltrue  
goto Lfalse
```

Ltrue:

```
Loan = Approved  
goto Lend
```

Lfalse:

```
Loan = Rejected
```

Lend:

Result: The loan is approved only when both salary and credit score satisfy the conditions.

Q3. Traffic Signal Control System

Given

switch(signal)

case 1: RED

case 2: YELLOW

case 3: GREEN

1. Intermediate Code Using Conditional Jumps

if signal == 1 goto L1

if signal == 2 goto L2

if signal == 3 goto L3

goto Lend

L1:

signal_output = RED

goto Lend

L2:

signal_output = YELLOW

goto Lend

L3:

signal_output = GREEN

goto Lend

Lend:

2. Jump Table Representation

For multiple cases, a jump table can be used:

index = signal - 1

goto JumpTable[index]

JumpTable:

L1

L2

L3

L1:

signal_output = RED

goto Lend

L2:

```
signal_output = YELLOW  
goto Lend
```

L3:

```
signal_output = GREEN
```

Lend:

3. Efficiency

A jump table is efficient when there are many consecutive case values because it can directly jump to the required case.

For only three cases, conditional jumps are simple. For a large number of cases, a jump table generally provides faster selection.

Q4. Online Payment Gateway

Functions

**validateCard()
processPayment()
generateReceipt()**

Assume:

**validateCard(card)
processPayment(amount)
generateReceipt(transactionID)**

1. Intermediate Code

**param card
call validateCard, 1**

**param amount
call processPayment, 1**

**param transactionID
call generateReceipt, 1**

The 1 represents the number of parameters passed.

2. Parameter Passing and Call/Return

Three-address representation:

**param card
t1 = call validateCard, 1**

**param amount
t2 = call processPayment, 1**

**param transactionID
t3 = call generateReceipt, 1**

Here:

- **param places an argument for the procedure.**
- **call invokes the procedure.**
- **t1, t2, and t3 store return values if any.**

3. Stack-Based Representation

The procedure call can be represented as:

**Push(card)
Call validateCard
Return**

**Push(amount)
Call processPayment
Return**

**Push(transactionID)
Call generateReceipt
Return**

Call Sequence

Caller



Push parameters



Call procedure



Procedure executes



Return result



Caller continues

The stack stores parameters, return addresses, and other information required during procedure execution.

Q5. Smart Home Automation System

Condition

The AC should turn ON when:

Temperature > 30 AND RoomOccupied

1. Boolean Expression

B = (Temperature > 30) AND RoomOccupied

Both conditions must be true.

2. Intermediate Code with Short-Circuit Evaluation

if Temperature > 30 goto L1

goto Loff

L1:

if RoomOccupied == true goto Lon

goto Loff

Lon:

AC_status = ON

goto Lend

Loff:

AC_status = OFF

Lend:

Short-Circuit Evaluation

The second condition is checked only if:

Temperature > 30

is true.

For example:

Temperature = 25

The first condition is false, so the system immediately goes to Loff.

There is no need to check RoomOccupied.

3. Backpatching

Initially:

```
if Temperature > 30 goto _  
goto _
```

```
if RoomOccupied goto _  
goto _
```

After backpatching:

```
if Temperature > 30 goto L1  
goto Loff
```

```
L1:  
if RoomOccupied goto Lon  
goto Loff
```

The incomplete jump targets are filled with the appropriate labels.

4. Final Assignment

Lon:

```
AC_status = ON  
goto Lend
```

Loff:

```
AC_status = OFF
```

Lend:

Final Flow

Temperature > 30?

Yes | No



L1 → **AC OFF**



Room Occupied?

Yes | No



AC ON **AC OFF**

